XJTLU Entrepreneur College (Taicang) Cover Sheet

Module code and Title	**DTS406TC Natural Language Processing**
School Title	**School of Artificial Intelligence and Advanced Computing**
Assignment Title	**Coursework 1 (CW 1)**
Submission Deadline	**5 pm China time (UTC+8 Beijing) on** 22/May/2026
Final Word Count	**3000**
If you agree to let the university use your work anonymously for teaching and learning purposes, please type **"yes"** here.	

I certify that I have read and understood the University's Policy for dealing with Plagiarism, Collusion and the Fabrication of Data (available on Learning Mall Online). With reference to this policy I certify that:

- My work does not contain any instances of plagiarism and/or collusion.
  My work does not contain any fabricated data.

By uploading my assignment onto Learning Mall Online, I formally declare that all of the above information is true to the best of my knowledge and belief.

Scoring – For Tutor Use	
Student ID	2575700 2575704 2575741

Stage of Marking	**Marker Code**	**Learning Outcomes Achieved (F/P/M/D) (please modify as appropriate)**			**Final Score**
		A	**B**	**C**	
1st Marker – red pen					
Moderation – green pen	**IM Initials**	The original mark has been accepted by the moderator (please circle as appropriate):			Y / N
		Data entry and score calculation have been			Y

<table>
<tr><td colspan="3"></td><td colspan="4">checked by another tutor (please circle):</td><td></td></tr>
<tr><td colspan="3">2nd Marker if needed – green pen</td><td></td><td colspan="2"></td><td></td><td></td></tr>
<tr><td colspan="3">For Academic Office Use</td><td colspan="5">Possible Academic Infringement (please tick as appropriate)</td></tr>
<tr><td>Date Received</td><td>Days late</td><td>Late Penalty</td><td colspan="3">☐ Category A</td><td colspan="2" rowspan="5">Total Academic Infringement Penalty (A,B, C, D, E, Please modify where necessary)
_____</td></tr>
<tr><td rowspan="4"></td><td rowspan="4"></td><td rowspan="4"></td><td colspan="3">☐ Category B</td></tr>
<tr><td colspan="3">☐ Category C</td></tr>
<tr><td colspan="3">☐ Category D</td></tr>
<tr><td colspan="3">☐ Category E</td></tr>
</table>

Policy

The report should be submitted in PDF format. The word count limit of the report is 3,000. All code must be written in Python. It should be well-structured, easy to read, and thoroughly documented. Each function should include comments explaining its purpose and functionality. Ensure that the code runs without errors. All required files should be included in a single zip file. A README file is needed to explain how to run the code and list any dependencies. The document must be submitted through Learning Mall Online to the appropriate drop box.

Importance

AI is permitted as a learning aid for your coursework, but it must not be used to auto-generate entire sections of code or the final report.

DTS406TC Natural Language Processing

Coursework 1

Document Topic Classification

Team Members

Jiacheng Gui	2575704
Junhao Feng	2575700
Xinyu Ren	2575741

Contents

1 Introduction

Document topic classification assigns a document to a predefined topic label according to its text. It is a core natural language processing task because large text collections cannot be organised or searched efficiently by manual labelling alone. In this coursework, the task is studied through two classification scenarios: community question answering and news topic classification.

The project compares traditional sparse-feature methods and neural representation methods. The two traditional models are Naive Bayes and a linear Support Vector Machine. The two neural methods are a Word2Vec-based classifier and a DistilBERT-based classifier. All four models are applied to both datasets using the same train, validation, and test splits for each dataset.

The evaluation focuses on precision, recall, and F1-score, with accuracy also reported as an additional summary metric. Macro averages are important because they show how well each model performs across labels rather than only reflecting the dominant class. The final analysis uses the recorded test-set results.

2 Literature Reviews

2.1 Literature Review by Jiacheng Gui

2.1.1 Overview and Applications

Document topic classification is a supervised NLP task that assigns documents to predefined topic labels. These documents can be news articles, community questions, academic abstracts, reviews, or social media posts. This task is useful because large text collections are difficult to organise manually. A typical pipeline includes document representation, classifier training, and evaluation on unseen data [1].

There are several real-life applications. First, news platforms can classify articles into topics such as sports, business, politics, or entertainment, which supports content navigation and recommendation. Second, academic databases can organise papers by research fields, which improves indexing and retrieval. Third, community question answering platforms can route user questions to suitable categories or experts, such as health, education, science, or finance.

2.1.2 Key Challenges

Document topic classification has several challenges. The first challenge is high-dimensional sparse representation. Traditional methods often use Bag-of-Words or TF-IDF features, where each word or n-gram may become one feature. This creates a large and sparse feature space. The second challenge is semantic ambiguity. A word can have different meanings in different contexts, and different words can express similar meanings. The third challenge is uneven topic structure, including topic overlap and label imbalance. Some documents contain signals from multiple topics, and imbalanced labels can reduce macro-level performance.

2.1.3 Traditional Methods

Naive Bayes is a common traditional baseline for text classification. It estimates the probability of each class from word occurrence features. The multinomial Naive Bayes model is often used for document classification because it is simple and efficient [2]. Its main advantage is fast training and easy implementation. However, it assumes that features are conditionally independent given the class label, which is not realistic for natural language.

Support Vector Machine is another strong traditional method. A linear SVM with TF-IDF features is effective for high-dimensional sparse text data [3]. Its advantage is that it can learn clear decision boundaries and often performs better than simple probabilistic baselines.

However, it depends heavily on feature engineering and cannot easily capture word order or deeper contextual meaning.

2.1.4 Deep Learning Approaches

A Word2Vec-based classifier uses dense word embeddings learned from text corpora to build document representations, for example by averaging word vectors before classification [4]. This method can capture some semantic similarity between words. However, average pooling loses word order, sequence information, and contextual meaning, so it may be weak for documents with complex topic signals.

BERT-based classification uses contextual representations from a pretrained Transformer model [5]. It can capture context-dependent meanings and is useful for ambiguous or informal texts. However, it requires more computation, longer training time, and careful control of sequence length. These training costs and sequence length limits make it less lightweight than traditional methods.

2.1.5 Summary

Overall, traditional methods are efficient and useful baselines, while deep learning methods usually provide stronger semantic or contextual representations, but require more computation and careful training. Therefore, using both traditional and neural methods provides a useful basis for comparing sparse lexical features, dense semantic embeddings, and contextual representations across different datasets.

2.2 Literature Review by Junhao Feng

2.2.1 Overview and Applications

Document topic classification is a foundational Natural Language Processing (NLP) task that assigns predefined thematic categories to unstructured textual data. By transforming unformatted text into structured metadata, classification models unlock advanced data mining capabilities. In real-world environments, this technology is deployed across high-stakes domains. First, in medical diagnostics, clinical NLP models extract tumor characteristics from pathology reports, aiding patient care and epidemiological research [6]. Second, academic resource systems utilize pattern classifiers to automatically generate metadata for Massive Open Online Courses (MOOCs) and categorize vast distance-learning repositories [7]. Third, the news industry relies on real-time classification pipelines to moderate content, evaluate publication trustworthiness, and filter misinformation, ensuring algorithmic accountability [8].

2.2.2 Key Challenges

Despite algorithmic advancements, the deployment of robust classification systems is hindered by data complexities. Class imbalance remains a critical obstacle, occurring when certain thematic categories dominate a dataset while minority classes are underrepresented. This asymmetry severely skews decision boundaries, causing predictive models to favor majority classes and perform poorly on rare occurrences [9]. Additionally, text datasets inherently suffer from high dimensionality and sparsity. Because natural language features an enormous global vocabulary, vector representations generate massive, mostly empty mathematical matrices, triggering computational inefficiencies and over-fitting [3]. Finally, semantic ambiguity restricts model reliability. Traditional frameworks treat words as isolated entities, failing to capture grammatical structures, polysemy, and dynamic contextual relationships, resulting in a profound loss of textual nuance [5].

2.2.3 Methodological Evolution

Historically, traditional machine learning established the framework for text categorization. The Bag-of-Words (BoW) model represents text as an unordered frequency matrix. While its primary advantages are exceptional computational efficiency and native interpretability, BoW suffers from an inability to capture sequential context, resulting in severe feature sparsity and blindness to out-of-vocabulary words [10]. To improve predictive performance, eXtreme Gradient Boosting (XGBoost) is applied as an ensemble decision-tree classifier. XGBoost boasts superior parameter efficiency, scalability, and robustness against class imbalance through aggressive regularization [11]. However, its core disadvantages include an inability to natively ingest raw text without external feature extractors and a high sensitivity to exhaustive hyperparameter tuning. To overcome traditional limitations, deep learning methodologies leverage continuous vector spaces. Long Short-Term Memory (LSTM) networks utilize specialized gating mechanisms to bypass the vanishing gradient problem inherent in recurrent neural networks. This provides the critical advantage of retaining temporal text structure and sequential dependencies [12]. Conversely, LSTMs are disadvantaged by their sequential processing nature, preventing hardware parallelization and causing extremely slow training cycles. Currently, Bidirectional Encoder Representations from Transformers (BERT) dominate the field by employing non-sequential self-attention mechanisms. BERT's primary advantage is its unprecedented bidirectional semantic comprehension, allowing for state-of-the-art accuracy. Nevertheless, these massive contextual advantages are offset by exorbitant computational costs, massive parameter counts, and high inferential latency [5].

2.3 Literature Review by Xinyu Ren

2.3.1 Overview and Applications

Document topic classification is a core supervised natural language processing (NLP) task that automatically assigns predefined topic labels to unstructured documents according to their semantic content [13]. It serves as a foundational technique for organizing, retrieving, and analyzing large-scale text corpora in both academic and industrial scenarios. Three representative real-world applications are widely adopted: news article categorization, which classifies news into politics, sports, entertainment, and technology for efficient content distribution; academic paper indexing, which organizes research works into disciplines such as computer science, medicine, and engineering to facilitate knowledge retrieval; and social media content tagging, which labels posts by topics like public health, environmental issues, or economic trends for content management and public opinion monitoring. These applications demonstrate the practical significance and wide applicability of document topic classification.

2.3.2 Key Challenges

Despite substantial progress, document topic classification still faces three fundamental challenges. First, *semantic ambiguity* caused by polysemy and context-dependent meanings hinders accurate topic judgment. Second, *high dimensionality and sparsity* of text features, due to large vocabulary sizes, increases computational complexity and degrades model generalization. Third, *domain adaptation* difficulty arises when training and test data differ in vocabulary, style, and topic distribution, leading to significant performance drops in cross-domain scenarios.

2.3.3 Traditional and Deep Learning Approaches

Traditional methods mainly include Naive Bayes and Support Vector Machines (SVM). Naive Bayes is computationally efficient, robust to high-dimensional data, and easy to implement, but it assumes word independence and ignores contextual information, limiting its performance on

complex texts. SVM, combined with TF-IDF features, delivers stable and accurate results on moderate datasets and exhibits strong generalization ability, yet it suffers from high computational costs when dealing with large-scale corpora.

Deep learning methods, exemplified by Long Short-Term Memory (LSTM) and Gated Recurrent Units (GRU), effectively capture sequential and contextual dependencies. LSTM mitigates the vanishing gradient problem and models long-range semantic dependencies well, achieving high accuracy on complex documents, but it requires substantial computational resources and longer training time. GRU simplifies the LSTM architecture with fewer parameters, accelerating training while maintaining competitive performance; however, it may lose subtle contextual details compared with LSTM.

3 Datasets and Preprocessing

The experiment deliberately uses two datasets with different classification scenarios to test the models under different text domains and label structures. Yahoo Answers Topics represents community question answering: short user questions and answers are assigned to 10 broad topics. AG News Sports vs Business represents binary news classification, where articles are separated into sports and business. The first setting is a multi-class informal Q&A task, while the second is a cleaner two-class news task.

Table 1: Full processed dataset statistics used for the experimental report.

Dataset	Scenario	Processed Rows	Labels	Train/Val/Test	Vocabulary	Avg. Length
Yahoo Answers Topics	Community Q&A	6,000	10	4,200/900/900	34,981	95.15
AG News Sports vs Business	News binary classification	59,952	2	41,966/8,993/8,993	51,655	54.89

All statistics in Table 1 are computed from the full processed dataset, meaning the train, validation, and test splits combined. This avoids reporting train-only statistics as if they described the whole dataset. Yahoo Answers contains 6,000 processed documents, and AG News contains 59,952 processed documents after cleaning and duplicate handling.

The preprocessing procedure applies lowercasing, tokenization, stopword removal, empty-text filtering, duplicate handling where necessary, and label standardisation. Traditional models and Word2Vec use the processed text representation. DistilBERT uses its own tokenizer during fine-tuning, so the text is not converted into sparse features before entering the model.

Table 2: Top frequent content words in the processed datasets after stopword and artefact filtering.

Rank	Yahoo Answers Word	Count	AG News Word	Count
1	like	1,894	new	17,020
2	know	1,643	oil	10,184
3	think	1,352	york	8,561
4	people	1,234	year	7,244
5	want	1,209	world	6,466
6	time	1,095	prices	6,307
7	good	1,066	stocks	6,291
8	need	1,025	game	5,872
9	help	959	company	5,097
10	way	841	win	5,078

Table 2 summarises frequent content words after excluding standard English stopwords, non-alphabetic tokens, and obvious corpus artefacts from the recorded word-frequency files. The

Yahoo Answers terms still reflect conversational question-answering language, whereas the AG News terms are more directly associated with sports and business reporting. Word frequency is therefore useful as a descriptive corpus summary, while the classification models rely on TF-IDF weighting, dense embeddings, or contextual token representations for prediction.

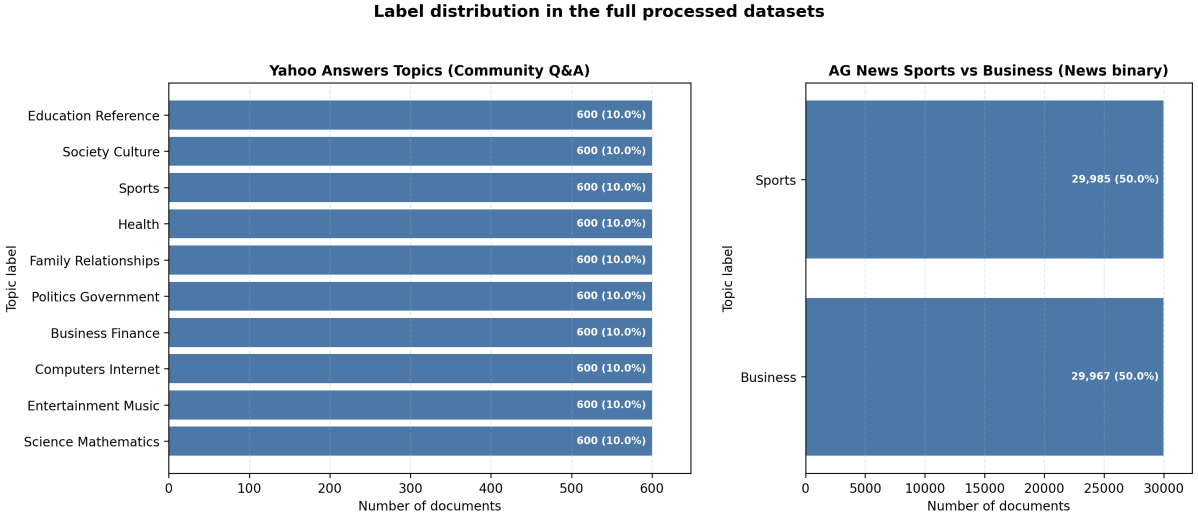

Figure 1: Label distributions computed from the full processed datasets.

Figure 1 shows that both datasets are balanced. In Yahoo Answers, each of the 10 topic labels has 600 documents, so macro-averaged metrics are meaningful for comparing performance across classes. In AG News, the sports and business classes are almost equally represented, with 29985 and 29967 documents respectively. Therefore, later performance differences are more likely to reflect task difficulty and model representation than severe label imbalance.

4 Algorithm Design

Pseudo-code in this section follows the actual project scripts. It is included for algorithm design explanation and is excluded from the report word count.

4.1 Naive Bayes

The Naive Bayes model uses TF-IDF unigram and bigram features with `MultinomialNB`. The vectorizer is fitted only on the training split, and the trained vectorizer is then applied to the test split.

Algorithm 1 TF-IDF + Multinomial Naive Bayes

Require: Training texts X_{train}, training labels y_{train}, test texts X_{test}
Ensure: Predicted labels $\hat{y}_{\text{test}}$ and evaluation metrics

1: Fit TF-IDF unigram/bigram vectorizer on X_{train} only.
2: Transform X_{train} into training vectors V_{train}.
3: Train `MultinomialNB` on $(V_{\text{train}}, y_{\text{train}})$.
4: Transform X_{test} using the fitted vectorizer to obtain V_{test}.
5: Predict test labels $\hat{y}_{\text{test}}$ with the trained classifier.
6: Compute precision, recall, F1-score, and accuracy.

The feature representation is sparse lexical evidence. The classifier is efficient and gives a clear baseline, but its conditional independence assumption limits its ability to model word

interactions.

4.2 Support Vector Machine

The SVM model uses TF-IDF unigram and bigram features with `LinearSVC`. A linear classifier is suitable for high-dimensional sparse text features.

Algorithm 2 TF-IDF + Linear Support Vector Machine

Require: Training texts X_{train}, training labels y_{train}, test texts X_{test}
Ensure: Predicted labels $\hat{y}_{\text{test}}$ and evaluation metrics
1: Fit TF-IDF unigram/bigram vectorizer on X_{train} only.
2: Transform X_{train} into training vectors V_{train}.
3: Train `LinearSVC` on $(V_{\text{train}}, y_{\text{train}})$.
4: Transform X_{test} using the fitted vectorizer to obtain V_{test}.
5: Predict test labels $\hat{y}_{\text{test}}$ with the trained SVM.
6: Compute precision, recall, F1-score, and accuracy.

The main component is the maximum-margin linear decision boundary. Compared with Naive Bayes, it does not model class-conditional word probabilities, but it often works well with sparse text features.

4.3 Word2Vec-based Classifier

The Word2Vec pipeline trains embeddings only on the training split. Each document is represented by the average of its in-vocabulary token vectors, and Logistic Regression performs final classification.

Algorithm 3 Self-trained Word2Vec + Average Pooling + Logistic Regression

Require: Training texts X_{train}, training labels y_{train}, test texts X_{test}
Ensure: Predicted labels $\hat{y}_{\text{test}}$ and evaluation metrics
1: Tokenize training texts into token lists T_{train}.
2: Train Word2Vec only on T_{train}.
3: **for** each document d in training and test texts **do**
4: Collect Word2Vec vectors for in-vocabulary tokens in d.
5: **if** no token in d is in vocabulary **then**
6: Use a zero vector as the document vector.
7: **else**
8: Average in-vocabulary word vectors into one document vector.
9: **end if**
10: **end for**
11: Train Logistic Regression on training document vectors and y_{train}.
12: Predict test labels $\hat{y}_{\text{test}}$ from test document vectors.
13: Compute precision, recall, F1-score, and accuracy.

The implementation uses `vector_size=100`, `window=5`, `min_count=2`, `sg=1`, and Logistic Regression with `max_iter=1000`. Its limitation is that average pooling removes word order and much of the local context.

4.4 BERT-based Classifier

The BERT-based model fine-tunes `distilbert-base-uncased` for sequence classification. Labels are encoded with `LabelEncoder`; predictions are decoded back to the original label strings

before saving.

Algorithm 4 DistilBERT Fine-tuning for Document Topic Classification

Require: Training texts X_{train}, training labels y_{train}, test texts X_{test}
Ensure: Predicted labels $\hat{y}_{\text{test}}$ and evaluation metrics
1: Encode string labels as integer class IDs.
2: Load `distilbert-base-uncased` tokenizer and sequence classification model.
3: Tokenize input text with maximum length 128, truncation, and padding.
4: Fine-tune the DistilBERT classifier on the training split.
5: Evaluate the fine-tuned model on the test split.
6: Decode predicted class IDs back to original label strings.
7: Compute precision, recall, F1-score, and accuracy.

DistilBERT provides contextual token representations, which helps with ambiguous or informal documents. The cost is higher training and inference time than the traditional models.

5 Implementation

All models follow the same experimental protocol. For each dataset, the prepared train, validation, and test splits are fixed before model training. The same split is used for all four models, so performance differences are not caused by different test examples. The validation split is kept for possible parameter checking, while the final reported metrics are computed on the test split.

For Naive Bayes and SVM, the TF-IDF vectorizer is fitted only on the training text. The fitted vectorizer is then reused to transform the test text. This prevents information from the test split leaking into the feature space. Both models use unigram and bigram features with a maximum vocabulary size of 25000. Naive Bayes applies a multinomial probabilistic classifier, while SVM uses a linear maximum-margin classifier.

For the Word2Vec-based classifier, embeddings are trained only on the training split. Each document is represented by the average of its in-vocabulary word vectors, and documents without in-vocabulary tokens receive a zero vector. A Logistic Regression classifier is then trained on these document vectors. This setup keeps the method simple and reproducible, but it also means that word order and context are not directly modelled.

The BERT-based classifier fine-tunes `distilbert-base-uncased` for sequence classification. Input text is tokenized with truncation and padding to a maximum length of 128. The default setup uses batch size 16, learning rate 2e-5, two training epochs, and GPU acceleration when available. Labels are encoded during training and decoded back to their original topic names for evaluation.

Evaluation is unified across all models. The reported measures are macro precision, macro recall, macro F1-score, and accuracy. Macro scores are emphasised because they treat each class equally. Reproducibility is supported by a fixed random seed of 42 for splitting, model initialization where applicable, and deterministic Word2Vec worker settings.

6 Results Analysis

Table 3 reports the test-set results for all model-dataset pairs. Within each dataset, models are sorted by macro F1-score because macro F1 balances precision and recall across classes. The best result for each dataset is highlighted in bold.

Table 3: Test performance by dataset, sorted by macro F1-score within each dataset.

Dataset	Model	Precision Macro	Recall Macro	F1 Macro	Accuracy
Yahoo Answers Topics	DistilBERT	**0.6764**	**0.6822**	**0.6750**	**0.6822**
Yahoo Answers Topics	Linear SVM	0.5740	0.5778	0.5736	0.5778
Yahoo Answers Topics	Naive Bayes	0.5936	0.5500	0.5383	0.5500
Yahoo Answers Topics	Word2Vec avg	0.4336	0.4433	0.4291	0.4433
AG News Sports vs Business	DistilBERT	**0.9945**	**0.9944**	**0.9944**	**0.9944**
AG News Sports vs Business	Linear SVM	0.9896	0.9895	0.9895	0.9895
AG News Sports vs Business	Word2Vec avg	0.9859	0.9859	0.9859	0.9859
AG News Sports vs Business	Naive Bayes	0.9845	0.9843	0.9843	0.9843

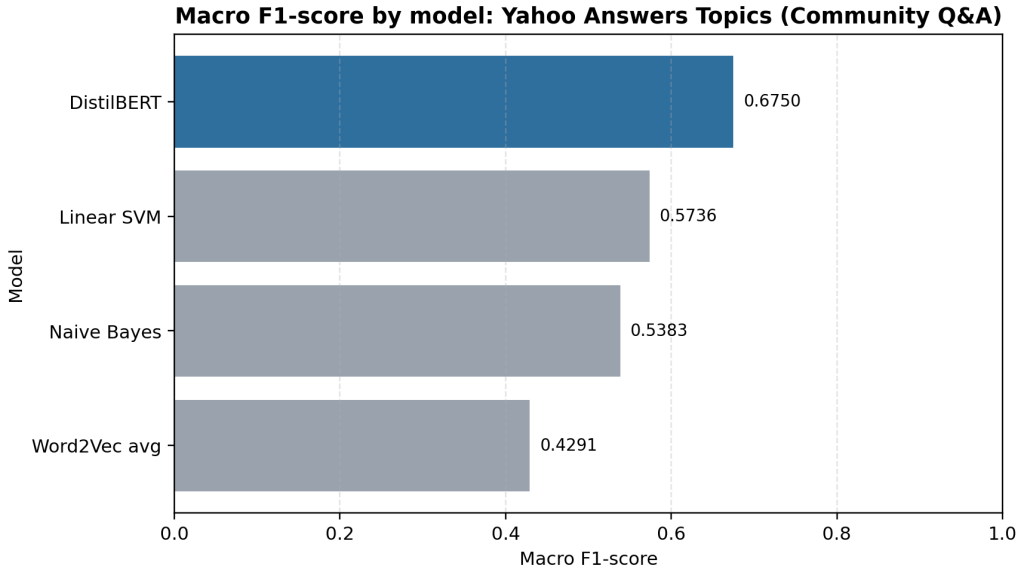

Figure 2: Macro F1-score comparison on Yahoo Answers Topics.

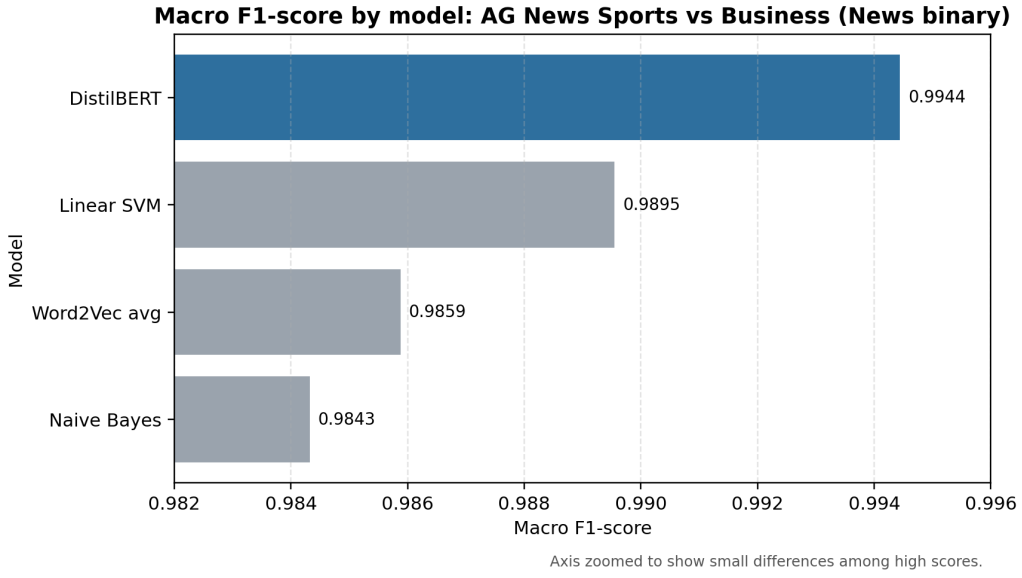

Figure 3: Macro F1-score comparison on AG News Sports vs Business. The axis is zoomed because all models score above 0.98.

Yahoo Answers Topics is clearly harder than AG News Sports vs Business. The best macro F1 on Yahoo Answers is 0.6750 from DistilBERT, while the best macro F1 on AG News is 0.9944. The gap is expected because Yahoo Answers has 10 labels, informal community text, and overlapping topic signals. A question may contain terms that are useful for several categories, and short user-generated text can omit context. In contrast, AG News is a binary news classification task where sports and business vocabulary is often more separable.

Precision, recall, and F1 show slightly different aspects of performance. On Yahoo Answers, DistilBERT has the highest macro precision (0.6764), recall (0.6822), and F1-score (0.6750), which means its advantage is not caused by only one metric. SVM has a better macro F1 than Naive Bayes, although Naive Bayes has higher macro precision. This suggests that Naive Bayes is more conservative for some classes, while SVM gives a better precision-recall balance. Word2Vec has the lowest precision, recall, and F1 on Yahoo Answers. Its average pooling representation removes word order and contextual interactions, which is harmful for a noisy multi-class Q&A dataset.

On AG News, all four models are strong across precision, recall, F1-score, and accuracy. DistilBERT is still the best model, but the margin over SVM is small. SVM, Naive Bayes, and Word2Vec all exceed 0.984 macro F1, showing that clear topic vocabulary can make traditional lexical features highly competitive. These high scores should not be over-generalised to all topic classification tasks. They mainly show that this binary news task is easier and more lexically separable than the Yahoo Answers setting.

Overall, the results support a task-dependent interpretation. Contextual fine-tuning is most useful for the harder multi-class Q&A dataset. Traditional TF-IDF methods remain strong when the task has clear lexical boundaries. Average Word2Vec document vectors are lightweight, but their loss of order and context makes them less reliable on complex topic structures.

7 Conclusion

This project compares four document topic classification methods on two different scenarios. The results show that dataset difficulty matters. DistilBERT is more effective for the complex 10-class Yahoo Answers Q&A dataset, where contextual meaning and informal text are important. For the binary AG News Sports vs Business dataset, traditional TF-IDF models are already very strong because the two topics have clearer lexical separation.

Word2Vec average pooling is simple and useful, but it loses word order and contextual detail. Future improvement could include stronger sentence-level embeddings, more careful hyperparameter tuning on the validation split, and additional error analysis by label.

References

[1] F. Sebastiani, "Machine learning in automated text categorization," *ACM Computing Surveys*, vol. 34, no. 1, pp. 1–47, 2002.

[2] A. McCallum and K. Nigam, "A comparison of event models for naive bayes text classification," in *AAAI Workshop on Learning for Text Categorization*, 1998, pp. 41–48.

[3] T. Joachims, "Text categorization with support vector machines: Learning with many relevant features," in *European Conference on Machine Learning*. Springer, 1998, pp. 137–142.

[4] T. Mikolov, K. Chen, G. Corrado, and J. Dean, "Efficient estimation of word representations in vector space," *arXiv preprint arXiv:1301.3781*, 2013.

[5] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, 2019, pp. 4171–4186.

[6] D. Demner-Fushman, N. Elhadad, and C. Friedman, "Natural language processing for health-related texts," in *Biomedical Informatics: Computer Applications in Health Care and Biomedicine*. Springer, 2021, pp. 241–272.

[7] H. Sebbaq and N. eddine El Faddouli, "Fine-tuned BERT model for large scale and cognitive classification of MOOCs," *International Review of Research in Open and Distributed Learning*, vol. 23, no. 2, pp. 170–190, 2022.

[8] K. Shu, A. Sliva, S. Wang, J. Tang, and H. Liu, "Fake news detection on social media: A data mining perspective," *ACM SIGKDD Explorations Newsletter*, vol. 19, no. 1, pp. 22–36, 2017.

[9] I. H. Karaman, G. Koksal, L. Eriskin, and S. Salihoglu, "A similarity-based oversampling method for multi-label imbalanced text data," *International Journal of Data Science and Analytics*, vol. 21, no. 1, p. 45, 2026.

[10] C. D. Manning, P. Raghavan, and H. Schutze, *Introduction to Information Retrieval*. Cambridge University Press, 2008.

[11] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016, pp. 785–794.

[12] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

[13] D. Jurafsky and J. H. Martin, "Speech and language processing," 2023, draft.